

* Multithreading in PYTHON :- *

Index:

⇒ Types of Applications.

a) process Based Applications.

b) Thread Based Applications.

⇒ purpose of multi threading.

⇒ Definition of Thread.

⇒ purpose of thread,

⇒ programming examples.

⇒ module name required for developing thread Based Applications (threading).

⇒ function in "threading" module

⇒ programming examples

⇒ Number of approaches to develop multi threading Applications.

a) By using functional programming.

b) By using oriented programming.

⇒ creating multiple programmer Defined Threads.

⇒ programming examples.

⇒ Dead lock occurrence

⇒ Thread synchronization.

⇒ implementation of thread synchronization for eliminating Dead lock occurrence

⇒ implementing of lock class.

⇒ programming examples.

*Types of Applications in multi threading.

→ The purpose of multithreading is that "to provide concurrent execution / simultaneous execution or parallel processing (executing all at once)."

→ In industry, we have two types of Applications/Languages. They are

- 1) Process Based Applications.
- 2) Thread Based Applications.

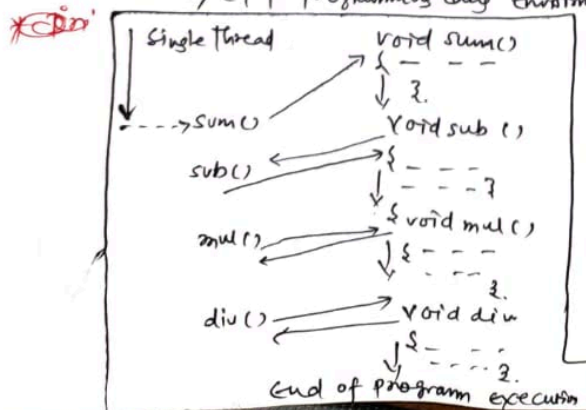
1) Process Based Applications:-

- process Based Applications contains single thread.
 - process Based Applications provides sequential execution.
 - process Based Application Takes more Execution time
 - process Based Applications are treated as heavy weight components.
- e.g) C, C++

2) Thread Based Applications:-

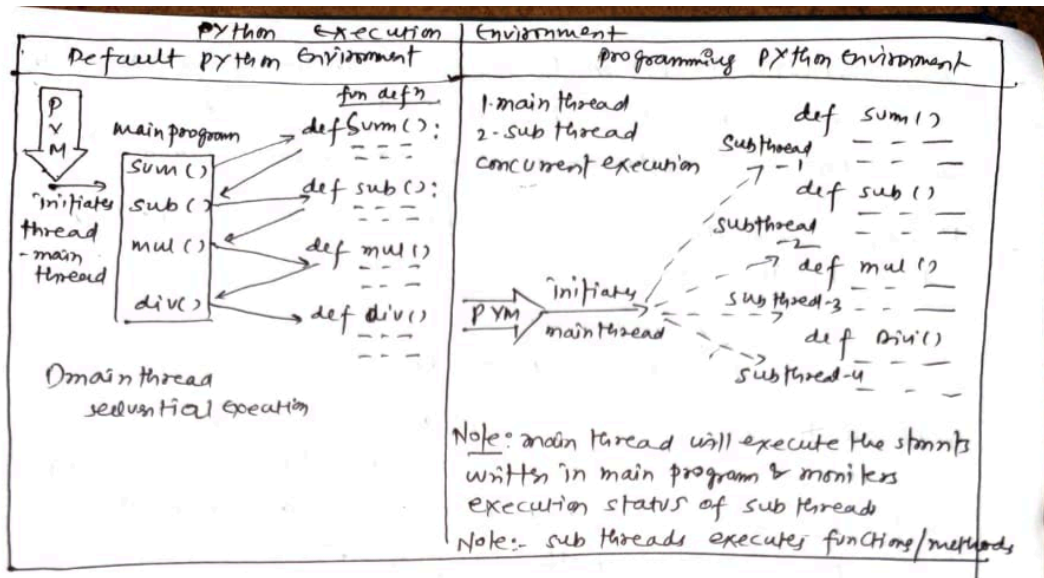
- Thread Based Applications contains by default single thread and programmatically we can create multiple threads.
 - Thread Based Application provides both sequential execution and concurrent execution
 - Thread Based Application takes less execution time
 - Thread Based Applications are treated as light weight components.
- e.g) Python, Java, C#.net ... etc.

C, C++ programming lang Environment



Process Based Appln

- Contains single thread
 - provides sequential execution
 - Takes more execution time
 - Treated as heavy comp components
- C, C++ ... etc. PBA



* Introduction to Thread Based Applications:-

- the purpose of multi threading is that "to provide concurrent / simultaneous execution / parallel execution".
- concurrent execution is nothing but executing the operations all at once.
- the Advantage of concurrent execution is that to get less execution time
- if a python program contains multiple threads then it is called multi threading program.

* Definition of thread

- A flow of control is called thread.
- the purpose of thread is that "to perform certain operation whose logic developed in functions/ methods concurrently".
- By default every python contains single thread and whose name is "main thread" and it provides sequential execution

→ Programmatically, in python program we can create multiple sub/child threads and whose purpose is that "to execute operations whose logic is written in functions / methods concurrently".

→ Hence, programmatically a python program contains two types of threads. They are:

a) main Thread.

b) sub/child threads.

a) Main thread is created/initiated by pvm, when program execution starts and the role of main thread is to execute main program statements and monitor the execution status of sub threads (if sub threads present)

b) the sub/child threads always executes operations whose logic is written in functions / methods concurrently.

program for finding name of main thread.

Default thread.

import threading

t = threading.current_thread()

print("Default name of thread =", t.name)

program for demonstrating non-thread based app (contains only main thread but not sub thread)

Non-threads Ex1)

import threading

def Hello():

print("Hello executed by {}".format(threading.current_thread().name))

def Hi():

print("Hi executed by {}".format(threading.current_thread().name))

def welcome():

print("welcome")

main program

```
print("Program execution started by: {}.".format(__name__))
print("Number of threads in this program =", threading.active_count())
print(" -- ")
hello()
print(" - - - ")
hi()
print(" ' - - ' ")
welcome()
print("welcome")
print("Program execution done.")
```

Main sub threads ex 1. py

ex 1) sab se pehle main program ke pahle print kare

create 3 sub threads.

```
t1 = threading.Thread(target=hello) # Here t1 is object of Thread - sub
t2 =                               h1 thread & its default name is
t3 =                               welcome thread-1
```

dispatch the sub threads.

```
t1.start()
t2.start()
t3.start()
print(" - - - ")
print("Program execution started by __name__")
```

Non threads ex 2

import threading, time

def square(lst):

for val in lst:

```
print("{} - - square({}) = {}".format(threading.currentThread().name,
val, val**2))
time.sleep(1)
```

def cube(lst):

for val in lst:

```
print("{} - - - cube -
time.sleep(1)
val**3))
```

```

# main program, bt = time.time()
print ("program Execution started by: {}".format(threading.currentThread().name))
print (" -- ")
lst = [12, 4, 15, -5, 14, 19, 25, 8]
squares(lst)
print (" -- ")
cubes(lst)
print (" -- ")
st = time.time()
print ("Time taken: {}".format(st - bt))

```

main subthread Ex-2.py

Sub thread by main program & char

main program

3. एक थ्रेड की मदद से

```

print ("Number of threads before sub threads start = ", threading.active_count())
print (" -- ")
lst of name

```

create 2 sub threads.

```

t1 = threading.Thread(target=squares, args=(lst))
t2 = threading.Thread(target=cubes, args=(lst))

```

t1.start()

t2.start()

```

print ("number of thread after sub threads start = ", threading.active_count())

```

join the sub threads.

t1.join()

t2.join()

```

print (" -- ")

```

```

print ("Number of threads after sub threads complete = ", threading.active_count())

```

```

print ("program Execution started by: {}".format(threading.currentThread().name))

```

et = time.time()

Baki same.

Day 89

* Module Name Required for Developing Thread Based Application :-

→ The module Name Required for Developing Thread Based Application is "threading".

⇒ The Threading module contains the following details :-

• module Name :- threading.

* Function Names in Threading module :-

1) Current_thread() :- This function is used for obtaining current thread which is under execution.

syntax :- Varname = threading.current_thread().
name

2) Active_Count() :- This function is used for obtaining number of Active threads (By Default 1)

syntax :- Varname = threading.Active_Count()

* Class Name-2 : Thread Class

→ Here 'Thread' is one of the pre-defined class in Threading module

⇒ The 'Thread' contains the following.

1) Thread(target, args) :

syntax :- Varname = threading.Thread(target = Function/method)
name, args = (val1, val2, ... val-n))

→ This constructor creates an object Thread class called sub thread and initialized with which sub thread targetted to execute which function where thread logic is defined and args represents the value(s) carrying from main program to function OR method.

if the function or method not taking any parameters, No need to specify args.

Ex) . t1 = threading.Thread(target = square, args = ([10, 20, 30, 40], [2, 3, 4, 5],))

Here `H` is called sub thread whose default name is `Thread-1`

2) start()

→ This function is used for Dispatching (sending to the targetted function) the sub threads.

syntax). `Subthreadobj.start()`

e.g). `t1.start()`.

3) setName(str)

→ syntax: `Subthread.setName("sub Thread Name")`

→ This function is used for setting the Name to sub Thread and it deprecated on the name of "name" attribute

→ Recommended syntax :- `Subthread.name = "sub Thread Name"`

4) getName()

→ syntax: `tname = Subthread.getName()`

→ This function is used for obtaining name of sub Thread and it deprecated on the name of "name" attribute.

→ recommended syntax: `tname = Subthread.name`

5) is_alive()

→ This function is used for omitting the execution status of sub threads.

syntax). `Subthreadobj.is_alive()`

→ This function returns True provided thread is running.

→ This function returns false provided thread is not running (Before execution, After execution).

6) join()

→ syntax :- `Subthreadobj.join()`

→ This function is used for making the main thread to wait until // Sub threads join with main thread

Program for obtaining default thread (main thread)

current thread.

```

ex1) import threading
t = threading.current_thread()
# t.name = t.getName() --- not recommended bcoz
# t.name = t.name           getName() is deprecated
# print("Default thread name =", t.name)      on the name of 'name' attribute
# print(" --- OR --- ")
t.name = threading.current_thread().name
  
```

Active Count

```

import threading
print("Default name of thread =", threading.current
      thread().name)
print("By default, Number of threads active =",
      threading.active_count())
  
```

Program for creating sub thread, setname and get name of the threads.

Subthread creation ex1)

```

import threading
def welcome(val):
    print(" - - - ")
    print("Name of sub thread in welcome() =", threading.
          current_thread().name)
    print("{} Good evening".format(val))
    print(" - - - ")
  
```

main program

Create sub thread

```
t1 = threading.Thread(target=welcome, args=("Rossum",))
```

```
print("Name of sub thread in main program before setting =", t1.name)
```

```
# t1.setName("Rajesh") --- not recommended bcoz setName() is
                        deprecated on "name" attribute.
```

```
t1.name = "Rajesh".
```

```
print("Name same (after) .
```

dispatch or send the sub threads to targetted function.

```
t1.start()
```

ex1). Subthread ExecStatus

Same as. print the bad time sleep (10).

main program

Create sub thread

print("name of main thread =", threading.current_thread().name)

print(" - - - ")

print("Number of threads in this program before sub threads =",
threading.active_count())

t1 = threading.Thread(target=welCome, args=("ossum",))

print("Execution status of sub thread before start", t1.is_alive())

Dispatch.

t1.start()

print("Execution status of sub thread after start =", t1.is_alive())

print("Number of threads in this program during execution
=", threading.active_count())

print("Execution status of sub thread after completion =",
t1.is_alive())

print(" - - - ")

Q. Develop a thread based Application which will generate
1 to n numbers after each and every second

NumGen ex1)

import threading, time

def generate(n):

print("Name of the threading generate() =",
threading.current_thread().name)

n = int("How many numbers you want to generate")

if (n <= 0):

print("{} invalid input".format(n))

else:

print(" - - - ")

print("Numbers within: {}".format(n))

print(" - - - ")

for i in range(1, n+1):

print("\t value of i = {}".format(i))

time.sleep(1)

print(" - - - ")

main program

n = int(input("Enter How many numbers you want to generate:"))

t1 = threading.Thread(target = generate, args = (n,)) # sub thread creation

t1.start()

ex2)

print ke niche

→ it likhna hai

→ ye karta hain kisi ki

ex3) Numbers aop ex1).

jitc def generate self,

Bay change krke class Numbers: ki sthiti ki ex2

main program ki Bar Add karna hai.

no = numbers() Baaki sab as it is

target = Numbers().generate

①

ex4) . no = numbers()

t1 = threading.Thread(target = no.generate)

②

t1.start()

ex5)

③ . n = int(input("How many number you want to generate"))

no = Numbers()

t1 = threading.Thread(target = no.generate, args = (n,))

t1.start()

ex6)

4)

class Numbers:

def __init__(self, n):

self.n = n

def generate(self):

main program

n = input vtr.

no = Numbers(n) # object creation calls parametrized constructor.

t1 = threading.Thread(target = no.generate)

t1.start()

* Number of approaches to develop Thread Based Applications:-

→ In Python, to develop Thread Based Applications, we have two approaches. They are

- 1) By using functional programming.
- 2) By using object oriented programming.

1) By using functional programming:-

- step 1) import Threading module & other modules if required
- step 2) Define function(s) where it contains operational logic executed by sub thread.
- step 3) Create sub thread(s).
- step 4) Dispatch the sub threads by using start()

2) By using object oriented programming.

- step 1) import Threading module and other modules if required
- step 2) Define programmer-Defined Class
- step 3) Define a programmer-Defined method(s) where it contains operational logic executed by sub thread.
- step 4) Create sub thread(s)
- step 5) Dispatch the sub threads by using start()

Day 85 * Synchronization in multi threading (OR) * locking concept in threading

- when multiple threads are operating/working on the same resource (function/method) then by default we get dead lock result/race condition/wrong results/non-thread safety result.
- To overcome this dead lock problems we must apply the concept of synchronization.
- The Advantage of synchronization concept is that to avoid dead lock result and provides thread safety result.
- In python programming, we can obtain synchronization concept by using locking and un-locking concept.

* Step for implementing synchronization concept:

- 1) obtain/create an object of lock class, which is present in multithreading module

syntax: `lockobj = threading.Lock()`

- 2) to obtain the lock on the sharable resources, we must use `acquire()`

syntax: `lockobj.acquire()`

→ once current object acquire the lock, other thread objects are made wait until current thread object releases the lock.

- 3) to un-lock the sharable resource/current object, we must use `release()`

syntax: `lockobj.release()`

→ once current object releases the lock, other objects are permitted into sharable resource. This process of acquiring and releasing the lock will be continued until all the thread objects.

Program for generating Even and odd numbers separately by two threads functional mechanism

Gen odd threads (o)

import threading, time

def oddnums(n):

if (n <= 0):
print("{} --> {} is invalid input".format(threading.current_thread().name, n))

else:

for i in range(1, n+1, 2):

print("{} -- odd numbers: {}".format(threading.current_thread().name, i))

time.sleep(1)

def evennums(n):

if (n <= 0):

→ same.

else:

for i in range(2, n+1, 2):

→ same.

time.sleep(1)

main program

Create sub threads.

t1 = threading.Thread(target=oddnums, args=int(input("Enter How many odd numbers u want generate: ")))

t2 = 1 "same" "evennums"

t1.start()

t2.start()

ex2). ex1 ki jaisi ki sab class Numbers: ki jaisi ki class, OOP.

jitni Bar (self, n) aur jaisi ki oddnums or evennums ki jaisi

jitni # main program ki.

Numbers().oddnum } Baaki sab same
Numbers().evennum }

Program for demonstrating multiple threads enters into same functions and raises Dead lock / Race Condition

ex 1) import threading, time # Non-synfun ex 1

```
def multable(n):
    if(n <= 0):
        same as example 1.
    else:
        print("-" * 50)
        print("{} -- mul table for {}".format("name", n))
        print("-" * 50)
        for i in range(1, 11):
            print("{} * {} = {}".format(n, i, n * i))
            time.sleep(1)
        print("-" * 50)
```

main program

create multiple sub threads.

```
t1 = threading.Thread(target=multable, args=(10,))
t2 = threading.Thread(target=multable, args=(12,))
t3 = threading.Thread(target=multable, args=(9,))
t4 = threading.Thread(target=multable, args=(19,))
```

dispatch the threads.

```
t1.start()
t2.start()
t3.start()
t4.start()
```

ex 2) Non-synfun ex 1.

sub class table:

```
class Table:
    def multable(self, n):
```

```
# main program
t = Table().multable(sub t1, t2, t3, t4).
```

synfun ex 1).

```
import threading
l = threading.Lock() # steps 1 of 11 line 11.
# ex 1 of copy last A.
l.release() -- # steps
```

main program of B.

```
# create an object of lock class
l = threading.Lock() -- # step 12,
```

some
non-synfun ex 1
example
lenat 11

syn op ex1

Same likhna hai. ex2). nm-syn-op-ex1 ke jaisa hai.

$\left. \begin{array}{l} L.acquire() \\ L.release() \\ l = threading.Lock() \end{array} \right\}$
 ये तीनी syn fun के तरह इस्तेमाल करना है।

syn op ex2

class Table: ~~class~~

② class method

def getlock(cls):

cls.L = threading.Lock() # step 1

def multable(self, n):

table.L.acquire() # step 2

if ke baad same ex1 ke jaisa

table.L.release()

main program

Table.getlock()

Baaki sub same

syn op ex2

class Table:

② classmethod

def getlock(cls):

cls.L = threading.Lock()

def __init__(self, n):

self.n = n

def multable(self):

table.L.acquire()

if (self.n <= 0) { Baaki sub same }

print("It's ", (self.n, 1, self.n * 1))

main program

table.getlock()

target = table(10).multable

it likhna hai same

```
In [2]: #Program for Demonstrating Getting Default Thread
#DefaultThreadName
import threading
tname=threading.current_thread().name
print("Default Thread Name:",tname)
print("Number of Active Threads by Default=", threading.active_count())
```

Default Thread Name: MainThread
Number of Active Threads by Default= 6

```
In [3]: #Program for Demonstrating How Single Thread will execute all the functions and
#DefaultThreadFlow
```



```

import threading
def hello():
    print("hello()--Executed by=",threading.current_thread().name)
def welcome():
    print("welcome()--Executed by=",threading.current_thread().name)
def hi():
    print("hi()--Executed by=",threading.current_thread().name)

#main Program
print("Program Execution Started by :",threading.current_thread().name)
print("-----")
hello()
print("-----")
welcome()
print("-----")
hi()
print("-----")
print("Program Execution Ended by :",threading.current_thread().name)

```

Program Execution Started by : MainThread

hello()--Executed by= MainThread

welcome()--Executed by= MainThread

hi()--Executed by= MainThread

Program Execution Ended by : MainThread

In [4]: *#Program for Deminstrating obtaining Default Thread*
#CurrentThreadEx
import threading
t=threading.current_thread()
print("Default Thread Name=",t.name)
print("-----")
tname=threading.current_thread().name
print("Default Thread Name=",tname)

Default Thread Name= MainThread

Default Thread Name= MainThread

In [5]: *#Program for Deminstrating obtaining number of active threads*
#ActiveThreadCountEx
import threading
tname=threading.current_thread().name
print("Default Thread Name=",tname)
noact=threading.active_count()
print("Number of Active Threads=",noact)

Default Thread Name= MainThread

Number of Active Threads= 6

In [6]: *#Program for Demonstrating Execution of single thread and Execution time*
#SingleThreadExecutionEx
import threading,time
def squares(lst):
 for val in lst:
 print("{}---Square({})={}".format(threading.current_thread().name,
time.sleep(1)

```

def cubes(lst):
    for val in lst:
        print("{}---cube({})={}".format(threading.current_thread().name,
        time.sleep(1)

#Main Program
bt=time.time()
print("Program Execution Started:{}".format(threading.current_thread().name))
lst=[4,12,6,18,-5,16,25,-9,2]
squares(lst)
print("-----")
cubes(lst)
print("Program Execution Ended:{}".format(threading.current_thread().name))
et=time.time()
print("Total Execution of Time this program--single threaded={}".format(et-bt))

```

Program Execution Started:MainThread

MainThread---Square(4)=16

MainThread---Square(12)=144

MainThread---Square(6)=36

MainThread---Square(18)=324

MainThread---Square(-5)=25

MainThread---Square(16)=256

MainThread---Square(25)=625

MainThread---Square(-9)=81

MainThread---Square(2)=4

MainThread---cube(4)=64

MainThread---cube(12)=1728

MainThread---cube(6)=216

MainThread---cube(18)=5832

MainThread---cube(-5)=-125

MainThread---cube(16)=4096

MainThread---cube(25)=15625

MainThread---cube(-9)=-729

MainThread---cube(2)=8

Program Execution Ended:MainThread

Total Execution of Time this program--single threaded=18.017945051193237

In [7]: *#Program for Demonstrating How Multiple Sub Thread will execute all the function*
#SubThreadFlow

```

import threading
def hello():
    print("hello()--Executed by=",threading.current_thread().name)
def welcome():
    print("welcome()--Executed by=",threading.current_thread().name)
def hi():
    print("hi()--Executed by=",threading.current_thread().name)

#main Program
print("Program Execution Started by :",threading.current_thread().name)
print("-----")
#create Sub Thread t1--name Thread-1
t1=threading.Thread(target=hello)
t1.name="KVR"
t1.start()
print("-----")
#create Sub Thread t2name Thread-2
t2=threading.Thread(target=welcome)
t2.start()

```

```

print("-----")
#create Sub Thread t3name Thread-3
t3=threading.Thread(target=hi)
t3.start()
print("-----")
t1.join()
t2.join()
t3.join()
print("Program Execution Ended by :",threading.current_thread().name)

```

Program Execution Started by : MainThread

hello()--Executed by= KVR

welcome()--Executed by= Thread-4 (welcome)

hi()--Executed by= Thread-5 (hi)

Program Execution Ended by : MainThread

```

In [8]: #Program for Demonstrating Execution of Multiple threads and Execution time
#MultipleThreadExecutionEx
import threading,time
def squares(lst):
    for val in lst:
        print("{}---Square({})={}".format(threading.current_thread().name,
            val,squares(val)))
        time.sleep(1)

def cubes(lst):
    for val in lst:
        print("{}---cube({})={}".format(threading.current_thread().name,
            val,cubes(val)))
        time.sleep(1)

#Main Program
bt=time.time()
print("Program Execution Started:{}".format(threading.current_thread().name))
lst=[4,12,6,18,-5,16,25,-9,2]
#create a sub thread t1 for executing Squares
t1=threading.Thread(target=squares,args=(lst,))
#create a sub thread t2 for executing Cubes
t2=threading.Thread(target=cubes,args=(lst,))
t1.start()
t2.start()
#Make MainThread to Wait until sub threads Joins with MainThread
t1.join()
t2.join()
print("Execution Status of t1 after join=",t1.is_alive())
print("Execution Status of t2 after join=",t2.is_alive())
print("Program Execution Ended:{}".format(threading.current_thread().name))
et=time.time()
print("Total Execution of Time this program--single threaded={}".format(et-bt))

```

```

Program Execution Started:MainThread
Thread-6 (squares)---Square(4)=16
Thread-7 (cubes)---cube(4)=64
Thread-6 (squares)---Square(12)=144Thread-7 (cubes)---cube(12)=1728

Thread-7 (cubes)---cube(6)=216Thread-6 (squares)---Square(6)=36

Thread-6 (squares)---Square(18)=324
Thread-7 (cubes)---cube(18)=5832
Thread-6 (squares)---Square(-5)=25
Thread-7 (cubes)---cube(-5)=-125
Thread-6 (squares)---Square(16)=256
Thread-7 (cubes)---cube(16)=4096
Thread-6 (squares)---Square(25)=625
Thread-7 (cubes)---cube(25)=15625
Thread-6 (squares)---Square(-9)=81
Thread-7 (cubes)---cube(-9)=-729
Thread-6 (squares)---Square(2)=4
Thread-7 (cubes)---cube(2)=8
Execution Status of t1 after join= False
Execution Status of t2 after join= False
Program Execution Ended:MainThread
Total Execution of Time this program--single threaded=9.014727115631104

```

```

In [9]: #Thread Based Application for 1 to n numbers where n is +ve after every second
#NumberGenerationFunEx1
import threading,time
def numbergenerate(n):
    if(n<=0):
        print("{}---{} Invalid Input:".format(threading.current_thread(),n))
    else:
        print("-----")
        print("{}-->Numbers within:{}".format(threading.current_thread(),n))
        print("-----")
        for i in range(1,n+1):
            print("\tVal of i={}".format(i))
            time.sleep(1)
        print("-----")

#Main Program
n=int(input("Enter How Many Numbers u want to generate:"))
t1=threading.Thread(target=numbergenerate,args=(n,))
t1.name="number generator"
t1.start()

```

```

-----
number generator-->Numbers within:5
-----

```

```

    Val of i=1
    Val of i=2
    Val of i=3
    Val of i=4
    Val of i=5
-----

```

```

In [10]: #Thread Based Application for 1 to n numbers where n is +ve after every second
#NumberGenerationFunEx2
import threading,time
def numbergenerate():
    try:

```



```

n=int(input("Enter How Many Numbers u want to generate:"))
if(n<=0):
    print("{}---> {} Invalid Input:".format(threading.current_
else:
    print("-----")
    print("{}-->Numbers within:{}".format(threading.current_
    print("-----")
    for i in range(1,n+1):
        print("\tVal of i={}".format(i))
        time.sleep(0.25)
    print("-----")
except ValueError:
    print("Don't Enter alnums, strs and symbols")

#Main Program

t1=threading.Thread(target=numergenerate)
t1.name="number genarator"
t1.start()

```

```

-----
number genarator-->Numbers within:8
-----

```

```

Val of i=1
Val of i=2
Val of i=3
Val of i=4
Val of i=5
Val of i=6
Val of i=7
Val of i=8
-----

```

```

In [11]: #Thread Based Application for 1 to n numbers where n is +ve after every second
#NumberGenerationOpsEx1
import threading,time
class Numbers:
    def numergenerate(self): # Instance Method-----
        try:
            n=int(input("Enter How Many Numbers u want to generate:"))
            if(n<=0):
                print("{}---> {} Invalid Input:".format(threadin
            else:
                print("-----")
                print("{}-->Numbers within:{}".format(threading.
                print("-----")
                for i in range(1,n+1):
                    print("\tVal of i={}".format(i))
                    time.sleep(0.25)
                print("-----")
        except ValueError:
            print("Don't Enter alnums, strs and symbols")

#Main Program
no=Numbers() # Object Creation specially for calling Instance Method
t1=threading.Thread(target=no.numergenerate)
t1.name="number genarator"
t1.start()

```

```
-----
number genarator-->Numbers within:8
-----
```

```
Val of i=1
Val of i=2
Val of i=3
Val of i=4
Val of i=5
Val of i=6
Val of i=7
Val of i=8
-----
```

```
In [12]: #Thread Based Application for 1 to n numbers where n is +ve after every second
#NumberGenerationOopsEx2
import threading,time
class Numbers:
    def numbergenerate(self,n): # Instance Method-----
        if(n<=0):
            print("{}--> {} Invalid Input:".format(threading.currentThread().name,n))
        else:
            print("-----")
            print("{}-->Numbers within:{}".format(threading.currentThread().name,n))
            print("-----")
            for i in range(1,n+1):
                print("\tVal of i={}".format(i))
                time.sleep(0.25)
            print("-----")

#Main Program
n=int(input("Enter How Many Numbers u want to generate:"))
t1=threading.Thread(target=Numbers().numbergenerate,args=(n,))
t1.name="number genarator"
t1.start()
#Numbers().numbergenerate is called Name-Less Object Creation
```

```
-----
number genarator-->Numbers within:8
-----
```

```
Val of i=1
Val of i=2
Val of i=3
Val of i=4
Val of i=5
Val of i=6
Val of i=7
Val of i=8
-----
```

```
In [13]: #Thread Based Application for 1 to n numbers where n is +ve after every second
#NumberGenerationOopsEx3
import threading,time
class Numbers:
    def __init__(self,n):
        self.n=n

    def numbergenerate(self): # Instance Method-----
        if(self.n<=0):
            print("{}--> {} Invalid Input:".format(threading.currentThread().name,self.n))
        else:
```

```

print("-----")
print("{}-->Numbers within:{}".format(threading.
print("-----")
for i in range(1,self.n+1):
    print("\tVal of i={}".format(i))
    time.sleep(0.25)
print("-----")

#Main Program
n=int(input("Enter How Many Numbers u want to generate:"))
no=Numbers(n)# Object Creation---Calls Constructor
t1=threading.Thread(target=no.numbergenerate)
t1.name="number genarator"
t1.start()
#Numbers().numbergenerate is called Name-Less Object Creation

```

```

-----
number genarator-->Numbers within:8
-----

```

```

Val of i=1
Val of i=2
Val of i=3
Val of i=4
Val of i=5
Val of i=6
Val of i=7
Val of i=8
-----

```

```

In [14]: #Thread Based Application for 1 to n numbers where n is +ve after every second
#NumberGenerationOpsEx4
import threading,time
class Numbers:
    def __init__(self,n):
        self.n=n

    def numbergenerate(self): # Instance Method-----
        if(self.n<=0):
            print("{}--> {} Invalid Input:".format(threadin

        else:
            print("-----")
            print("{}-->Numbers within:{}".format(threading.
            print("-----")
            for i in range(1,self.n+1):
                print("\tVal of i={}".format(i))
                time.sleep(0.25)
            print("-----")

#Main Program
t1=threading.Thread(target=Numbers(int(input("Enter How Many Numbers u want to g
t1.name="number genarator"
t1.start()
#Numbers().numbergenerate is called Name-Less Object Creation

```

 number generator-->Numbers within:8

Val of i=1
 Val of i=2
 Val of i=3
 Val of i=4
 Val of i=5
 Val of i=6
 Val of i=7
 Val of i=8

```
In [15]: #Program for Generating Odd and Even Numbers by using Multiple Threads
#OddEvenThreadsOops1
import threading,time
class OddNums:
    def odd(self,n):
        if(n<=0):
            print("{}--->{} Is Invalid Input".format(threading.current_thread().name,n))
        else:
            for i in range(1,n+1,2):
                print("{}--->Odd:{}".format(threading.current_thread().name,i))
                time.sleep(1)

class EvenNums:
    def even(self,n):
        if(n<=0):
            print("{}--->{} Is Invalid Input".format(threading.current_thread().name,n))
        else:
            for i in range(2,n+1,2):
                print("{}--->Even:{}".format(threading.current_thread().name,i))
                time.sleep(1)

#Main Program
on=int(input("Enter How Many Odd Numbers u want:"))
en=int(input("Enter How Many Even Numbers u want:"))
t1=threading.Thread(target=OddNums().odd,args=(on,))
t2=threading.Thread(target=EvenNums().even,args=(en,))
t1.name="ODD"
t2.name="EVEN"
t1.start()
t2.start()
```

ODD--->Odd:1
 EVEN--->Even:2
 EVEN--->Even:4ODD--->Odd:3
 EVEN--->Even:6

```
In [16]: #Program for Generating Odd and Even Numbers by using Multiple Threads
#OddEvenThreadsOops2
import threading,time
class EvenOddNums:
    def odd(self,n):
        if(n<=0):
            print("{}--->{} Is Invalid Input".format(threading.current_thread().name,n))
        else:
            for i in range(1,n+1,2):
                print("{}--->Odd:{}".format(threading.current_thread().name,i))
                time.sleep(1)
```



```

        time.sleep(0.25)

    def even(self,n):
        if(n<=0):
            print("{}--->{} Is Invalid Input".format(threading.curre
        else:
            for i in range(2,n+1,2):
                print("{}--->Even:{}".format(threading.current_t
                time.sleep(0.25)

#Main Program
on=int(input("Enter How Many Odd Numbers u want:"))
en=int(input("Enter How Many Even Numbers u want:"))

t1=threading.Thread(target=EvenOddNums().odd,args=(on,))

t2=threading.Thread(target=EvenOddNums().even,args=(en,))
t1.name="ODD"
t2.name="EVEN"
t1.start()

t2.start()

```

```

ODD--->Odd:1
EVEN--->Even:2
ODD--->Odd:3
ODD--->Odd:5

```

In [17]: *#Program for Showing Dead Lock Result*

```

#Non-SyncFunEx1
import threading,time
def table(n):
    if(n<=0):
        print("{}--->{}-->Is Invalid Input".format(threading.current_thr
    else:
        print("Mul Table for :{}".format(n))
        for i in range(1,11):
            print("{}-->{} x {} = {}".format(threading.curre
            time.sleep(1)
        print("-----")

#Main program
t1=threading.Thread(target=table,args=(7,))
t2=threading.Thread(target=table,args=(17,))
t3=threading.Thread(target=table,args=(-9,))
t4=threading.Thread(target=table,args=(19,))
t1.start()
t2.start()
t3.start()
t4.start()

```

```

Mul Table for :7
Thread-18 (table)-->7 x 1 = 7
Mul Table for :17
Thread-19 (table)-->17 x 1 = 17
Thread-20 (table)--->-9-->Is Invalid Input
Mul Table for :19
Thread-21 (table)-->19 x 1 = 19
Thread-18 (table)-->7 x 2 = 14
Thread-19 (table)-->17 x 2 = 34
Thread-21 (table)-->19 x 2 = 38
Thread-18 (table)-->7 x 3 = 21
Thread-19 (table)-->17 x 3 = 51
Thread-21 (table)-->19 x 3 = 57
Thread-18 (table)-->7 x 4 = 28
Thread-19 (table)-->17 x 4 = 68
Thread-21 (table)-->19 x 4 = 76
Thread-18 (table)-->7 x 5 = 35
Thread-19 (table)-->17 x 5 = 85
Thread-21 (table)-->19 x 5 = 95
Thread-18 (table)-->7 x 6 = 42
Thread-19 (table)-->17 x 6 = 102
Thread-21 (table)-->19 x 6 = 114
Thread-21 (table)-->19 x 7 = 133Thread-19 (table)-->17 x 7 = 119
Thread-18 (table)-->7 x 7 = 49

Thread-18 (table)-->7 x 8 = 56Thread-21 (table)-->19 x 8 = 152
Thread-19 (table)-->17 x 8 = 136

Thread-21 (table)-->19 x 9 = 171Thread-19 (table)-->17 x 9 = 153

Thread-18 (table)-->7 x 9 = 63
Thread-19 (table)-->17 x 10 = 170Thread-21 (table)-->19 x 10 = 190

Thread-18 (table)-->7 x 10 = 70
-----
-----

```

```

In [18]: #Program for Showing Dead Lock Result
#Non-SyncOOPsEx1
import threading,time
class MulTable:
    def table(self,n):
        if(n<=0):
            print("{}--->{}-->Is Invalid Input".format(threading.cur
        else:
            print("Mul Table for :{}".format(n))
            for i in range(1,11):
                print("{}-->{} x {} = {}".format(threadi
                time.sleep(1)
            print("-----")

#Main program
t1=threading.Thread(target=MulTable().table,args=(7,))
t2=threading.Thread(target=MulTable().table,args=(17,))
t3=threading.Thread(target=MulTable().table,args=(-9,))
t4=threading.Thread(target=MulTable().table,args=(19,))
t1.start()
t2.start()

```

```
t3.start()
t4.start()
```

```
Mul Table for :7
Thread-22 (table)-->7 x 1 = 7
Mul Table for :17
Thread-23 (table)-->17 x 1 = 17
Thread-24 (table)--->-9-->Is Invalid Input
Mul Table for :19
Thread-25 (table)-->19 x 1 = 19
Thread-22 (table)-->7 x 2 = 14
Thread-23 (table)-->17 x 2 = 34
Thread-25 (table)-->19 x 2 = 38
Thread-22 (table)-->7 x 3 = 21
Thread-23 (table)-->17 x 3 = 51
Thread-25 (table)-->19 x 3 = 57
Thread-22 (table)-->7 x 4 = 28
Thread-23 (table)-->17 x 4 = 68
Thread-25 (table)-->19 x 4 = 76
Thread-22 (table)-->7 x 5 = 35
Thread-23 (table)-->17 x 5 = 85
Thread-25 (table)-->19 x 5 = 95
Thread-22 (table)-->7 x 6 = 42
Thread-23 (table)-->17 x 6 = 102
Thread-25 (table)-->19 x 6 = 114
Thread-22 (table)-->7 x 7 = 49
Thread-23 (table)-->17 x 7 = 119
Thread-25 (table)-->19 x 7 = 133
Thread-22 (table)-->7 x 8 = 56
Thread-23 (table)-->17 x 8 = 136
Thread-25 (table)-->19 x 8 = 152
Thread-22 (table)-->7 x 9 = 63
Thread-23 (table)-->17 x 9 = 153
Thread-25 (table)-->19 x 9 = 171
Thread-22 (table)-->7 x 10 = 70
Thread-23 (table)-->17 x 10 = 170
Thread-25 (table)-->19 x 10 = 190
-----
-----
-----
```

```
In [19]: #Program for Showing Thread Safety Result by applying Synchronization (dead Loc
#SyncFunEx1
import threading,time
def table(n):
    L.acquire() # Step-2
    if(n<=0):
        print("{}--->{}-->Is Invalid Input".format(threading.current_thr
    else:
        print("Mul Table for :{}".format(n))
        for i in range(1,11):
            print("{}-->{} x {} = {}".format(threading.curre
            time.sleep(1)
        print("-----")
    L.release() # Step-3

#Main program
L=threading.Lock() # Here L is called an object Lock Class--Step-1
#Create sub threads by targetting Same Function
t1=threading.Thread(target=table,args=(7,))
```

```

t2=threading.Thread(target=table,args=(17,))
t3=threading.Thread(target=table,args=(-9,))
t4=threading.Thread(target=table,args=(19,))
t1.start()
t2.start()
t3.start()
t4.start()

```

Mul Table for :7

```

Thread-26 (table)-->7 x 1 = 7
Thread-26 (table)-->7 x 2 = 14
Thread-26 (table)-->7 x 3 = 21
Thread-26 (table)-->7 x 4 = 28
Thread-26 (table)-->7 x 5 = 35
Thread-26 (table)-->7 x 6 = 42
Thread-26 (table)-->7 x 7 = 49
Thread-26 (table)-->7 x 8 = 56
Thread-26 (table)-->7 x 9 = 63
Thread-26 (table)-->7 x 10 = 70
-----

```

Mul Table for :17

```

Thread-27 (table)-->17 x 1 = 17
Thread-27 (table)-->17 x 2 = 34
Thread-27 (table)-->17 x 3 = 51
Thread-27 (table)-->17 x 4 = 68
Thread-27 (table)-->17 x 5 = 85
Thread-27 (table)-->17 x 6 = 102
Thread-27 (table)-->17 x 7 = 119
Thread-27 (table)-->17 x 8 = 136
Thread-27 (table)-->17 x 9 = 153
Thread-27 (table)-->17 x 10 = 170
-----

```

Thread-28 (table)--->-9-->Is Invalid Input

Mul Table for :19

```

Thread-29 (table)-->19 x 1 = 19
Thread-29 (table)-->19 x 2 = 38
Thread-29 (table)-->19 x 3 = 57
Thread-29 (table)-->19 x 4 = 76
Thread-29 (table)-->19 x 5 = 95
Thread-29 (table)-->19 x 6 = 114
Thread-29 (table)-->19 x 7 = 133
Thread-29 (table)-->19 x 8 = 152
Thread-29 (table)-->19 x 9 = 171
Thread-29 (table)-->19 x 10 = 190
-----

```

In [20]: *#Program for Showing Thread Safety Result by applying Synchronization (dead Loc #SyncOOPsEx1*

```

import threading,time
class MulTable:
    def table(self,n):
        L.acquire()
        if(n<=0):
            print("{}--->{}-->Is Invalid Input".format(threading.cur
        else:
            print("Mul Table for :{}".format(n))
            for i in range(1,11):
                print("{}-->{} x {} = {}".format(threadi
                time.sleep(1)
            print("-----")

```

```

        L.release()

#Main program
L=threading.Lock()
t1=threading.Thread(target=MulTable().table,args=(7,))
t2=threading.Thread(target=MulTable().table,args=(-17,))
t3=threading.Thread(target=MulTable().table,args=(9,))
t4=threading.Thread(target=MulTable().table,args=(19,))
t1.start()
t2.start()
t3.start()
t4.start()

```

```

Mul Table for :7
Thread-30 (table)-->7 x 1 = 7
Thread-30 (table)-->7 x 2 = 14
Thread-30 (table)-->7 x 3 = 21
Thread-30 (table)-->7 x 4 = 28
Thread-30 (table)-->7 x 5 = 35
Thread-30 (table)-->7 x 6 = 42
Thread-30 (table)-->7 x 7 = 49
Thread-30 (table)-->7 x 8 = 56
Thread-30 (table)-->7 x 9 = 63
Thread-30 (table)-->7 x 10 = 70
-----
Thread-31 (table)--->-17-->Is Invalid Input
Mul Table for :9
Thread-32 (table)-->9 x 1 = 9
Thread-32 (table)-->9 x 2 = 18
Thread-32 (table)-->9 x 3 = 27
Thread-32 (table)-->9 x 4 = 36
Thread-32 (table)-->9 x 5 = 45
Thread-32 (table)-->9 x 6 = 54
Thread-32 (table)-->9 x 7 = 63
Thread-32 (table)-->9 x 8 = 72
Thread-32 (table)-->9 x 9 = 81
Thread-32 (table)-->9 x 10 = 90
-----
Mul Table for :19
Thread-33 (table)-->19 x 1 = 19
Thread-33 (table)-->19 x 2 = 38
Thread-33 (table)-->19 x 3 = 57
Thread-33 (table)-->19 x 4 = 76
Thread-33 (table)-->19 x 5 = 95
Thread-33 (table)-->19 x 6 = 114
Thread-33 (table)-->19 x 7 = 133
Thread-33 (table)-->19 x 8 = 152
Thread-33 (table)-->19 x 9 = 171
Thread-33 (table)-->19 x 10 = 190
-----

```

In [21]: *#Program for Showing Thread Safety Result by applying Synchronization (dead Lock)*
#SyncOOPsEx2

```

import threading,time
class MulTable:
    @classmethod
    def getLock(cls):
        cls.L=threading.Lock() # Here L is called Class Level Variable

    def table(self,n):

```



```

        MulTable.L.acquire()
        if(n<=0):
            print("{}--->{}-->Is Invalid Input".format(threading.cur
        else:
            print("Mul Table for :{}".format(n))
            for i in range(1,11):
                print("{}-->{} x {} = {}".format(threadi
                time.sleep(1)
            print("-----")
        MulTable.L.release()

```

#Main program

MulTable.getLock() # Calling Class Level Method

```

t1=threading.Thread(target=MulTable().table,args=(7,))
t2=threading.Thread(target=MulTable().table,args=(-17,))
t3=threading.Thread(target=MulTable().table,args=(9,))
t4=threading.Thread(target=MulTable().table,args=(19,))
t1.start()
t2.start()
t3.start()
t4.start()

```

Mul Table for :7

```

Thread-34 (table)-->7 x 1 = 7
Thread-34 (table)-->7 x 2 = 14
Thread-34 (table)-->7 x 3 = 21
Thread-34 (table)-->7 x 4 = 28
Thread-34 (table)-->7 x 5 = 35
Thread-34 (table)-->7 x 6 = 42
Thread-34 (table)-->7 x 7 = 49
Thread-34 (table)-->7 x 8 = 56
Thread-34 (table)-->7 x 9 = 63
Thread-34 (table)-->7 x 10 = 70

```

Thread-35 (table)--->-17-->Is Invalid Input

Mul Table for :9

```

Thread-36 (table)-->9 x 1 = 9
Thread-36 (table)-->9 x 2 = 18
Thread-36 (table)-->9 x 3 = 27
Thread-36 (table)-->9 x 4 = 36
Thread-36 (table)-->9 x 5 = 45
Thread-36 (table)-->9 x 6 = 54
Thread-36 (table)-->9 x 7 = 63
Thread-36 (table)-->9 x 8 = 72
Thread-36 (table)-->9 x 9 = 81
Thread-36 (table)-->9 x 10 = 90

```

Mul Table for :19

```

Thread-37 (table)-->19 x 1 = 19
Thread-37 (table)-->19 x 2 = 38
Thread-37 (table)-->19 x 3 = 57
Thread-37 (table)-->19 x 4 = 76
Thread-37 (table)-->19 x 5 = 95
Thread-37 (table)-->19 x 6 = 114
Thread-37 (table)-->19 x 7 = 133
Thread-37 (table)-->19 x 8 = 152
Thread-37 (table)-->19 x 9 = 171
Thread-37 (table)-->19 x 10 = 190

```

```

In [22]: #Program for Showing Reversevation System
#ReservationFunEx1
import threading,time
def reservation(nos):
    L.acquire()
    global totseats
    if(nos>totseats):
        print("Hi {}, {} are Not Available-try for next time".format(thr
time.sleep(2)
    else:
        totseats=totseats-nos
        print("Hi {}, {} Seats Reserved-Hpy Journey".format(threading.cu
        print("Available Seats={}".format(totseats))
        time.sleep(2)
        if(totseats==0):
            print("Train is Full")
    L.release()

#Main Program
L=threading.Lock()
totseats=11
print("Total Number of Seats:{}".format(totseats))
p1=threading.Thread(target=reservation,args=(3,))
p2=threading.Thread(target=reservation,args=(4,))
p3=threading.Thread(target=reservation,args=(6,))
p4=threading.Thread(target=reservation,args=(2,))
p5=threading.Thread(target=reservation,args=(3,))
p6=threading.Thread(target=reservation,args=(1,))
#dispatch the Threads (Passengers)
p1.start()
p2.start()
p3.start()
p4.start()
p5.start()
p6.start()

```

```

Total Number of Seats:11
Hi Thread-38 (reservation), 3 Seats Reserved-Hpy Journey
Available Seats=8
Hi Thread-39 (reservation), 4 Seats Reserved-Hpy Journey
Available Seats=4
Hi Thread-40 (reservation), 6 are Not Available-try for next time
Hi Thread-41 (reservation), 2 Seats Reserved-Hpy Journey
Available Seats=2
Hi Thread-42 (reservation), 3 are Not Available-try for next time
Hi Thread-43 (reservation), 1 Seats Reserved-Hpy Journey
Available Seats=1

```

```

In [23]: #Program for Showing Reversevation System
#ReservationOOPsEx1
import threading,time
class Train:
    @classmethod
    def getLock(cls):
        cls.L=threading.Lock()
        cls.totseats=10 # Here L and totseats are called Class Level Va
        print("Total Number of Seats:{}".format(cls.totseats))

    def __init__(self,nos):
        self.nos=nos

```

```

def reservation(self):
    Train.L.acquire()
    if(self.nos>Train.totseats):
        print("Hi {}, {} are Not Available-try for next time".format(self.nos, Train.totseats))
        time.sleep(2)
    else:
        Train.totseats=Train.totseats-self.nos
        print("Hi {}, {} Seats Reserved-Hpy Journey".format(self.nos, Train.totseats))
        print("Available Seats={}".format(Train.totseats))
        time.sleep(2)
        if(Train.totseats==0):
            print("Train is Full")
    Train.L.release()

#Main Program
Train.getLock()
p1=threading.Thread(target=Train(3).reservation)
p2=threading.Thread(target=Train(4).reservation)
p3=threading.Thread(target=Train(6).reservation)
p4=threading.Thread(target=Train(2).reservation)
p5=threading.Thread(target=Train(3).reservation)
p6=threading.Thread(target=Train(1).reservation)
#dispatch the Threads (Passengers)
p1.start()
p2.start()
p3.start()
p4.start()
p5.start()
p6.start()

```

Total Number of Seats:10
 Hi Thread-44 (reservation), 3 Seats Reserved-Hpy Journey
 Available Seats=7
 Hi Thread-45 (reservation), 4 Seats Reserved-Hpy Journey
 Available Seats=3
 Hi Thread-46 (reservation), 6 are Not Available-try for next time
 Hi Thread-47 (reservation), 2 Seats Reserved-Hpy Journey
 Available Seats=1
 Hi Thread-48 (reservation), 3 are Not Available-try for next time
 Hi Thread-49 (reservation), 1 Seats Reserved-Hpy Journey
 Available Seats=0
 Train is Full

In [24]: *#Banking00psEx.py*

```

import threading,time
class Bank:
    @classmethod
    def getLock(cls):
        cls.K=threading.Lock()
        cls.accbal=15000
    def withdraw(self,wamt):
        Bank.K.acquire()
        if(wamt>Bank.accbal):
            print("Hi {}, INR {} Is Not Available--Check Bounce".format(self, wamt))
            time.sleep(1)
        else:
            Bank.accbal=Bank.accbal-wamt
            print("Hi {}, INR {} Is Deposit--Verify".format(self, wamt))
            time.sleep(2)

```

```
        print("Available Bal:{}".format(Bank.accbal))
    Bank.K.release()

#Main Program
Bank.getLock()
c1=threading.Thread(target=Bank().withdraw,args=(5000,))
c2=threading.Thread(target=Bank().withdraw,args=(11000,))
c3=threading.Thread(target=Bank().withdraw,args=(8000,))
c4=threading.Thread(target=Bank().withdraw,args=(5000,))
c5=threading.Thread(target=Bank().withdraw,args=(2000,))
c6=threading.Thread(target=Bank().withdraw,args=(1000,))
for obj in [c1,c2,c3,c4,c5,c6]:
    obj.start()
```

```
Hi Thread-50 (withdraw), INR 5000 Is Deposit--Verify
Available Bal:10000
Hi Thread-51 (withdraw), INR 11000 Is Not Available--Check Bounce
Hi Thread-52 (withdraw), INR 8000 Is Deposit--Verify
Available Bal:2000
Hi Thread-53 (withdraw), INR 5000 Is Not Available--Check Bounce
Hi Thread-54 (withdraw), INR 2000 Is Deposit--Verify
Available Bal:0
Hi Thread-55 (withdraw), INR 1000 Is Not Available--Check Bounce
```

In []: